Server-Sent Events

HTTP server push with Node.js and Express

CSC 667/867 — Internet Applications

Polling vs. SSE

Polling

GET /status repeatedly

Most responses may contain no change.

Server-Sent Events

GET /events once

The server streams changes as they occur.

One HTTP request; one open response



- Server → browser: SSE events in a continuous HTTP response.
- Browser → server: separate HTTP requests, such as `fetch()`.
- No WebSocket upgrade.

SSE wire format

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache

id: 41
event: notice
data: {"text":"Room 12 cleared"}

id: 42
event: notice
data: {"text":"Door unlocked"}
```

Blank line = end of an event. The HTTP response remains open.

Event stream fields

Field	Meaning
`data:`	Payload, always text; JSON is optional
`event:`	Named event; default is `message`
`id:`	Remember the last event ID
`retry:`	Reconnection delay in milliseconds
`:`	Comment, ignored by `EventSource`

```
: heartbeat

```

A comment can keep an idle connection active. **It is not event data.**

Express: setup

```
'use strict';
const express = require('express');
const path = require('node:path');

const app = express();
const clients = new Set();
let nextId = 0;

app.use(express.json({ limit: '1kb' }));
app.use(express.static(path.join(__dirname, 'public')));
```

`clients` contains the open HTTP responses of subscribed browsers.

Express: open an SSE stream

```
app.get('/events', (req, res) => {
  res.set({
    'Content-Type': 'text/event-stream; charset=utf-8',
    'Cache-Control': 'no-cache, no-transform',
    'X-Accel-Buffering': 'no'
  });
  res.flushHeaders();
  res.write('retry: 2000\n\n');
  clients.add(res);

  const heartbeat = setInterval(() => res.write(': heartbeat\n\n'), 15000);
  res.on('close', () => {
    clearInterval(heartbeat);
    clients.delete(res);
  });
});
```

`res.write()` streams data; `res.end()` closes the response.

Express: publish to subscribers

```
app.post('/announce', (req, res) => {
  const text = req.body?.text;
  if (typeof text !== 'string' || !text.trim() || text.length > 200) {
    return res.status(400).json({ error: 'text must be 1-200 characters' });
  }

  const payload =
    `id: ${++nextId}\n` +
    'event: notice\n' +
    `data: ${JSON.stringify({ text })}\n\n`;
  for (const client of clients) client.write(payload);
  res.json({ id: nextId });
});

app.listen(3000, () => console.log('http://localhost:3000'));
```

One `POST` produces an event for every connected subscriber.

Browser: receive events and publish

```
const source = new EventSource('/events');

source.addEventListener('notice', event => {
  const item = document.createElement('li');
  item.textContent = JSON.parse(event.data).text;
  document.querySelector('#feed').prepend(item);
});

document.querySelector('#send').onclick = () => fetch('/announce', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ text: 'Room 12 cleared' })
});
```

`EventSource` receives; `fetch()` sends.

Run and observe

```
npm install
node server.js
```

Open `http://localhost:3000` in two browser tabs and send an announcement.

Or use two terminals:

```
# Terminal A: observe streaming events
curl -N http://localhost:3000/events

# Terminal B: publish an event
curl -X POST http://localhost:3000/announce \
  -H 'Content-Type: application/json' \
  -d '{"text":"Room 12 cleared"}'
```

Disconnects and event recovery

- `EventSource` reconnects automatically after most connection failures.
- `retry: 2000` sets a 2-second reconnection delay.
- When an event has `id:`, the browser can send `Last-Event-ID` on reconnect.
- **IDs do not store events.** Replaying missed events requires server-side history.
- `source.close()` stops the client; HTTP 204 can stop reconnection.

Our demo broadcasts only to currently connected clients.

Deployment and scale

- **Proxies:** disable response buffering; allow long-lived streams.
- **Slow clients:** bound write buffers and handle backpressure.
- **Security:** authenticate and authorize subscriptions; native `EventSource` cannot set arbitrary request headers.
- **HTTP/1.1:** browsers commonly limit concurrent SSE connections per origin; HTTP/2 multiplexes streams.

One Express process: broadcast directly to its local clients.

Several processes: Redis Pub/Sub can distribute notifications; it does not provide durable replay.

Choose the communication model

Technique	Direction	Use case
Polling	Request / response	Occasional status updates
SSE	Server → browser	Notifications, progress, live feeds
WebSocket	Bidirectional	Games, chat, collaborative editing

SSE is one-way messaging over an ordinary HTTP response.

Check for understanding

1. Why must an SSE response remain open?
2. What does the blank line after `data:` do?
3. Does `: heartbeat` create a browser event?
4. Does automatic reconnection guarantee recovery of missed events?
5. Why might a multiplayer game prefer WebSockets?

References

- WHATWG HTML Living Standard: Server-sent events
- MDN: Using server-sent events
- StackPractices: SSE with Node.js and Express
- Stack Overflow: SSE in Express
- Baynes: Real-time Web Applications with SSE